

Scalable Online Bus Route Search & Matching System

Design a scalable and configurable public bus journey planning system that allows users to search for journeys between any two locations/stops and returns all feasible direct and multi-leg bus journeys, ranked according to configurable criteria such as travel time, transfers, fare, walking distance, waiting time and user preferences.

The important design idea is to separate:

1. **Transport network modeling** — stops, routes, trips, buses, schedules.
2. **Journey matching** — source → destination, direct and multi-leg journeys.
3. **Ranking** — fastest, fewest transfers, cheapest, least walking, etc.
4. **Configuration** — ranking weights, search rules, transfer rules, time windows.
5. **Extensibility** — future support for metro, train, cab, multimodal journeys, real-time GPS, dynamic

Algorithm: TIME_DEPENDENT_DIJKSTRA

Algorithm: RAPTOR

Functional requirements:

Journey 1

Bus A
Stop A → Stop D
Departure: 10:00
Arrival: 10:40
Fare: ₹30

Journey 2

Bus B
Stop A → Stop B
Transfer
Bus C
Stop B → Stop D
Departure: 10:15
Arrival: 11:00
Fare: ₹25

Non-Functional Requirements:

Millions of stops
Thousands of routes
Millions of scheduled trips
Millions of daily searches

Configurable

Business rules should not be hardcoded.

Maximum walking distance = 1 km

Maximum transfers = 2

Ranking:

Travel Time = 40%

Transfers = 25%

Fare = 20%

Walking Distance = 10%

Waiting Time = 5%

Core Domain Model

Stop Route Trip Bus Schedule Journey Leg	Stop ----- id name latitude longitude address zone status	Route ----- id routeNumber name operatorId direction status	RouteStop ----- routeId stopId sequence distanceFromStart	Bus --- id registrationNumber operatorId busType capacity accessibility status
Example:	STOP_1001 Name: Ameerpet Latitude: 17.4375 Longitude: 78.4483	Route 10A A → B → C → D → E	Route 10A Sequence Stop ----- 1 A 2 B 3 C 4 D 5 E This sequence is extremely important. It allows us to determine: Can bus travel from A → D? because: sequence(A) < sequence(D)	Bus: TS09AB1234 Type: AC Capacity: 45

Trip ----- This is one of the most important concepts. A route describes: Where the bus travels. A trip describes: When a specific service travels that route. Example: Route: 10A :A → B → C → D → E Trips: Trip 1 ⇒ 10A: 08:00 → 09:00 Trips: Trip 2 ⇒ 10A: 09:00 → 10:00 Trips: Trip 3 ⇒ 10A: 10:00 → 11:00	Journey ----- id departureTime arrivalTime totalDuration totalFare walkingDistance waitingTime numberOfTransfers legs[]
--	---

Stop Time

A trip needs arrival/departure information for every stop.

TripStopTime

tripId

stopId

sequence

arrivalTime

departureTime

Example:

Trip 101

Stop	Arrival	Departure
------	---------	-----------

A	08:00	08:02
B	08:10	08:11
C	08:20	08:21
D	08:35	08:36
E	08:50	08:52

This enables precise journey planning.

Journey Leg

A journey can contain multiple legs.

Journey

```
├── Leg 1
│   ├── Bus 10A
│   └── A → C
├── Transfer
└── Leg 2
    ├── Bus 20B
    └── C → D
```

Transfer

fromStop

toStop

walkingDistance

walkingTime

minimumTransferTime

Example:

Bus 10A arrives: 10:30

Bus 20B departs: 10:38

If minimum transfer time is: 5 minutes

Then: **8 minutes** >= 5 minutes ⇐ **VALID**

Transfer Constraints

Transfer configuration can contain:

minimumTransferTime

maximumTransferTime

maximumTransfers

maximumWalkingDistance

Example:

```
{
  "minimumTransferTime": 5,
  "maximumTransferTime": 30,
  "maximumTransfers": 2
}
```

Multi-Leg Search

For multi-leg searches, model the transport network as a graph.

Stop = Node ;

Bus connection = Edge

Example:

```
A ---- B ---- C
      /       \
      D ----- E
```

Each edge can contain:

travelTime

Fare

route

trip

departureTime

arrivalTime

Search Request

A search request should not simply be: source & destination
Instead:

```
{
  "source": {
    "stopId": "A"
  },
  "destination": {
    "stopId": "D"
  },
  "travelDate": "2026-09-20",
  "departureTime": "10:00",
  "preferences": {
    "maximumTransfers": 2,
    "maximumWalkingDistance": 1000,
    "maximumFare": 100
  }
}
```

Search Modes

The system should support several modes.

Direct Bus: A → B → C → D

User searches: A → D

Return:

Bus 10A
A → D

One Transfer

A → B → C
↓
D → E

Example:

Bus 10A: A → B → C
Transfer at C
Bus 20B: C → D → E

Multiple Transfers

A
↓
Bus 10A
↓
B
↓
Bus 20B
↓
C
↓
Bus 30C
↓
D

Configuration can determine whether this is allowed.

Search Engine

Search Request
↓
Validate Request
↓
Find Candidate Stops
↓
Find Candidate Routes
↓
Find Candidate Trips
↓
Build Direct Journeys
↓
Build Transfer Journeys
↓
Apply Constraints
↓
Calculate Journey Metrics
↓
Rank Journeys
↓
Return Results

Stage 1 - Validate Search

Source exists Destination exists Date valid Time valid Source != Destination	Also validate configuration: maximumTransfers >= 0 maximumWalkingDistance >= 0
--	---

Stage 2 - Source and Destination Resolution

The user may not select an exact bus stop. They may search: Ameerpet Or: 17.4375, 78.4483 Therefore introduce: Location Resolution Service It can return: Nearest stops	Example: User Location ↓ Spatial Search ↓ Nearest Stops Stop A - 150m Stop B - 300m Stop C - 650m	Spatial Index Possible implementations: PostGIS Geohash H3 S2 R-tree KD-tree
--	--	--

Stage 3 — Find Candidate Routes Suppose: Source = A Destination = D Find routes containing both:: Route 1: A B C D Route 2: A X Y D Route 3: A B D	Route Matching sourceSequence < destinationSequence Example: A = 2 D = 5 Reverse direction: D = 5 A = 2 2 < 5: Valid 5 > 2: Invalid
---	--

Stage 4 — Find Candidate Trips

After finding candidate routes:	Example:
Route	User departure: 10:00
↓	
Trips	Candidate trips:
	09:30
	09:50
Filter trips based on requested time.	10:05
	10:30
	11:00

Stage 5 — Build Direct Journey

Suppose:	Journey: A → D
Trip 100	Departure: 10:05
	Arrival: 10:40
	Duration: 35 minutes
A 10:05	
B 10:15	
C 10:25	
D 10:40	

Graph Search

Possible algorithms include:

BFS
Dijkstra
A*
Time-dependent Dijkstra
RAPTOR
CSA
Multi-criteria routing

For a realistic public-transport system, time-dependent routing algorithms are important because an edge's availability depends on the departure time.

Why Normal Dijkstra Is Not Enough

Suppose: A → B

Bus 1: 10:00 → 10:20

Bus 2: 10:15 → 10:25

The cost of the edge depends on:
Current time

Therefore transportation networks are time-dependent graphs.

The algorithm must consider:

Current arrival time



Next available bus



Waiting time



Travel time

Ranking Requirements

Possible criteria:

Travel Time
Waiting Time
Number of Transfers
Fare
Walking Distance
Bus Comfort
Bus Type
Operator Preference
Accessibility
Reliability
Crowding

Ranking Strategy

```
interface RankingCriterion {  
    double calculateScore(Journey journey);  
}
```

Implement:

```
TravelTimeCriterion  
TransferCriterion  
FareCriterion  
WalkingDistanceCriterion  
WaitingTimeCriterion
```

Weighted Ranking

Example configuration:

```
{  
  "criteria": [  
    { "name": "TRAVEL_TIME", "weight": 0.40 },  
    { "name": "TRANSFERS", "weight": 0.25 },  
    { "name": "FARE", "weight": 0.20 },  
    { "name": "WALKING_DISTANCE", "weight": 0.10 },  
    { "name": "WAITING_TIME", "weight": 0.05 }  
  ]  
}
```

Then:

```
Final Score =  
  Travel Score × 0.40  
+ Transfer Score × 0.25  
+ Fare Score × 0.20  
+ Walking Score × 0.10  
+ Waiting Score × 0.05
```

Normalize Scores

Different metrics have different units.

Example:

Duration = 40 minutes

Fare = ₹30

Walking = 500 meters

Transfers = 1

They cannot directly be added.

Therefore normalize:

0 → worst

1 → best

For example:

Travel Score = normalized travel time

Fare Score = normalized fare

Walking Score = normalized walking distance

Ranking Pipeline

Candidate Journeys

↓
Calculate Metrics

↓
Normalize Metrics

↓
Apply Criteria

↓
Calculate Weighted Score

↓
Sort

↓
Return Top N

Strategy Pattern

```
public interface RankingStrategy {  
    double score(Journey journey);  
}
```

Implementations:

```
public class FastestJourneyStrategy  
    implements RankingStrategy {  
  
    @Override  
    public double score(Journey journey) {  
        // calculate score  
    }  
}
```

Composite Ranking

```
public interface RankingEngine {  
    List<Journey> rank(  
        List<Journey> journeys,  
        RankingConfiguration configuration  
    );  
}
```

Implementation:

WeightedRankingEngine

Configuration-Driven Architecture:

Do not hardcode: `MAX_TRANSFERS = 2;`

Instead: `Configuration Service`

Store:

`SEARCH_MAX_TRANSFERS`

`MAX_WALKING_DISTANCE`

`MIN_TRANSFER_TIME`

`MAX_RESULTS`

`RANKING_WEIGHTS`

Configuration Example

search:
 maximumTransfers: 2
 maximumWalkingDistanceMeters: 1000
 maximumResults: 20

transfer:
 minimumMinutes: 5
 maximumMinutes: 30

ranking:
 criteria:
 travelTime:
 enabled: true
 weight: 0.40

 transfers:
 enabled: true
 weight: 0.25

 fare:
 enabled: true
 weight: 0.20

 walkingDistance:
 enabled: true
 weight: 0.10

 waitingTime:
 enabled: true
 weight: 0.05

User Preferences

System-level ranking and user-level preferences should be separate.

Example user:

User prefers: **Cheapest**

or

User prefers: **Fastest**

or

User prefers: **Least walking**

Therefore:

System Configuration
+
User Preferences
↓
Effective Ranking Configuration

Profiles:

FASTEST
CHEAPEST
FEWEST_TRANSFERS
LEAST_WALKING
BALANCED
CUSTOM

Ranking Profile

Example:

```
{
  "name": "FASTEST",
  "criteria": {
    "travelTime": 0.70,
    "waitingTime": 0.15,
    "transfers": 0.10,
    "walkingDistance": 0.05
  }
}
```

Another:

```
{
  "name": "CHEAPEST",
  "criteria": {
    "fare": 0.70,
    "travelTime": 0.15,
    "transfers": 0.10,
    "walkingDistance": 0.05
  }
}
```

Pareto Optimization

Consider:: Journey A : 30 minutes ₹50 2 transfers Journey B: 40 minutes ₹20 0 transfers Neither is objectively better in every dimension. The system can support Pareto-optimal journeys.	A journey is dominated if another journey is: No slower AND No more expensive AND No more transfers AND No more walking while being strictly better in at least one dimension. This can reduce thousands of candidates to a meaningful set.
---	---

Result Grouping

Instead of returning 100 nearly identical results:

Journey 1 Journey 2 Journey 3 ... Journey 100	group them: Fastest: 10:00 → 10:40 Cheapest: 10:15 → 11:00 Fewest Transfers: 10:05 → 10:45 Least Walking: 10:10 → 10:50
---	---

This gives users meaningful choices.

API Design

POST /api/v1/journeys/search

Request: <pre>{ "source": { "latitude": 17.4375, "longitude": 78.4483 }, "destination": { "latitude": 17.3850, "longitude": 78.4867 }, "date": "2026-09-20", "departureTime": "10:00", "rankingProfile": "BALANCED" }</pre>	Response: <pre>{ "searchId": "SEARCH-123", "journeys": [{ "departureTime": "10:05", "arrivalTime": "10:45", "durationMinutes": 40, "fare": 30, "transfers": 0, "walkingDistanceMeters": 300, "legs": [] }] }</pre>
--	---

Single Responsibility Principle

Do not create: **BusSearchService** that does everything.

Instead:

StopResolver
RouteMatcher
TripFinder
TransferFinder
JourneyBuilder
JourneyValidator
JourneyScorer
RankingEngine
JourneyResponseMapper

Open/Closed Principle

Suppose tomorrow we introduce: **CrowdingScore**

The existing ranking engine should not need major changes.

Add:

CrowdingCriterion

and register it.

Existing code remains unchanged.

Liskov Substitution Principle

Implementations should respect contracts.

For example: **RankingCriterion**

must always behave like a ranking criterion.

Do not create one implementation that unexpectedly returns: **null**

or changes the meaning of the score.

Interface Segregation

Avoid:

```
interface TransportService {  
    searchBus();  
    searchTrain();  
    searchMetro();  
    trackVehicle();  
    calculateFare();  
    reserveSeat();  
}
```

Prefer smaller interfaces:

RouteSearcher
TripSearcher
FareCalculator
VehicleTracker
SeatAvailabilityProvider

Factory Pattern

Use a factory when selecting algorithms.

For example: **RoutingAlgorithmFactory**

Configuration.

routingAlgorithm = **RAPTOR**
returns: **RaptorRoutingAlgorithm**.

Later:
routingAlgorithm = **CSA**
returns: **ConnectionScanAlgorithm**

Strategy Pattern

Excellent candidates:

Routing algorithm
Fare calculation
Ranking
Transfer calculation
Location resolution

Example:

RoutingStrategy

- └─ **DirectRouteStrategy**
- └─ **GraphRoutingStrategy**
- └─ **RaptorRoutingStrategy**
- └─ **MultimodalRoutingStrategy**

Specification Pattern

Search filters can use the Specification Pattern.

example:

MaximumFareSpecification
MaximumTransferSpecification
MaximumWalkingSpecification
OperatorSpecification
AccessibilitySpecification

Combine:

MaximumFare
AND
MaximumTransfers
AND
MaximumWalking

Chain of Responsibility

Candidate journey processing can use:

Validate
↓
Time Filter
↓
Transfer Filter
↓
Fare Filter
↓
Walking Filter
↓
Accessibility Filter

Each handler can reject invalid journeys.

Dynamic ETA

Introduce: EtaProvider:

```
interface EtaProvider {  
    Duration estimateArrival(Trip trip, Stop stop);  
}
```

Implementations:

ScheduledEtaProvider
RealtimeEtaProvider
TrafficAwareEtaProvider

Fare Calculation

```
interface FareCalculator {  
    Money calculate(Journey journey);  
}
```

Implementations:

FlatFareCalculator
ZoneFareCalculator
DistanceFareCalculator
DynamicFareCalculator

Calendar and Service Days

Trips should not simply have: **trip.startTime** because schedules depend on:

Monday
Tuesday
Weekend
Holiday
Special service

Create: **ServiceCalendar**

Example:

Route 10A

Monday-Friday:
Every 15 minutes

Saturday:
Every 30 minutes

Sunday:
Every 45 minutes

Stop Closure

A stop can become temporarily unavailable.

Model: **StopStatus**

ACTIVE
CLOSED
TEMPORARILY_CLOSED

The **journey planner** should automatically exclude closed stops.

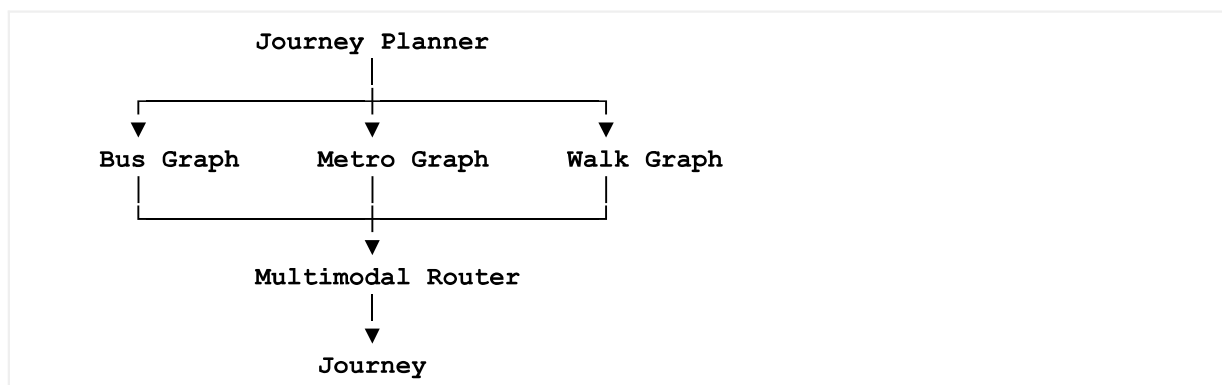
Admin System

Admin



Create Stop
Create Route
Add Route Stops
Create Trip
Configure Schedule
Configure Fare
Configure Ranking
Close Stop
Disable Route

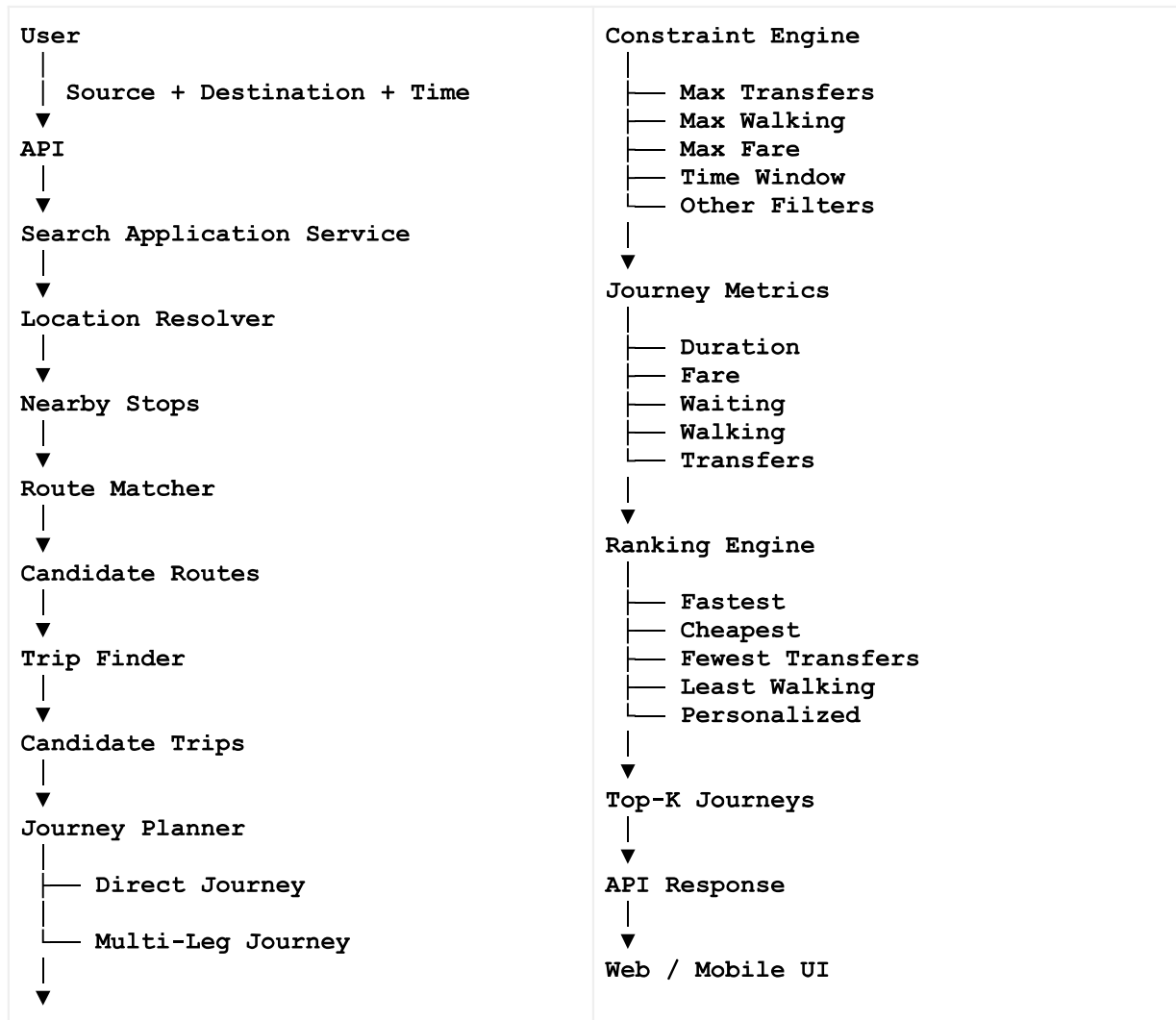
Multimodal Architecture



Core Interfaces

<pre>public interface StopResolver { List<StopCandidate> resolve(Location location); }</pre>	<pre>public interface RouteMatcher { List<RouteCandidate> findRoutes(Stop source, Stop destination); }</pre>
<pre>public interface TripFinder { List<Trip> findTrips(Route route, SearchCriteria criteria); }</pre>	<pre>public interface JourneyPlanner { List<Journey> plan(SearchRequest request); }</pre>
<pre>public interface JourneyFilter { boolean matches(Journey journey); }</pre>	<pre>public interface RankingCriterion { double score(Journey journey, RankingContext context); }</pre>
<pre>public interface RankingEngine { List<Journey> rank(List<Journey> journeys, RankingConfiguration configuration); }</pre>	<pre>public interface FareCalculator { Money calculate(Journey journey); }</pre>
<pre>public interface EtaProvider { Instant estimateArrival(Trip trip, Stop stop); }</pre>	

Final Search Flow



Make it a more generic Public Transport Journey Planning & Tracking System

The important distinction

A bus system typically looks like:

Stop → Route → Trip → Stop Time

A train system has similar concepts:

Station → Train Route → Train Trip →
Station Time

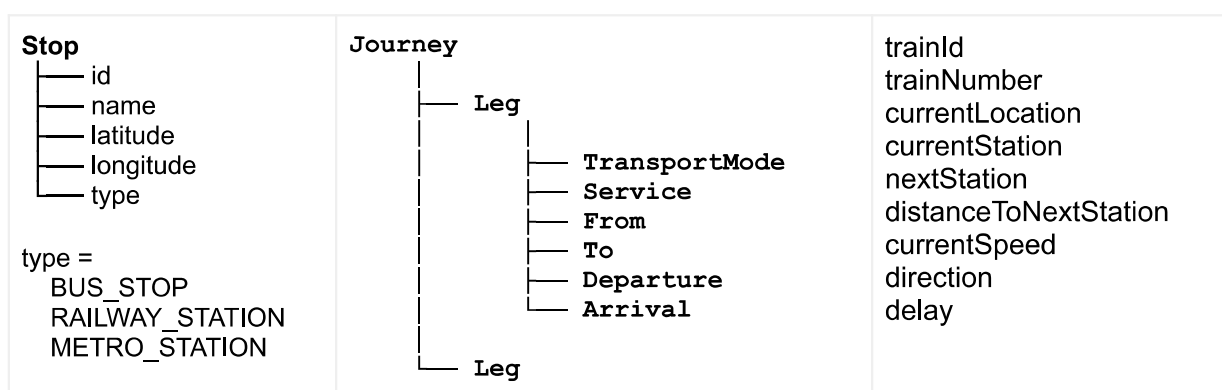
But trains introduce additional concepts that buses may not have:

Train	Delay
Train Number	Live Location
Station	ETA
Railway Route	Schedule
Train Run	Operational Day
Coach	Train Composition
Platform	

So your architecture should have a common transport abstraction, with mode-specific extensions.

Common Transport Model

Instead of:	Design:	Then::	For example:
Bus	TransportOperator	TransportMode	TransportStop
BusRoute	TransportRoute	— BUS	— BusStop
BusTrip	TransportTrip	— TRAIN	— RailwayStation
BusStop	TransportStop	— METRO	— MetroStation
	StopTime	— TRAM	
	Journey	— WALK	
	JourneyLeg	— BIKE	
	Transfer	— TAXI	



Train Schedule

Train 12701			This is conceptually the same as: Bus → StopTime
Station	Arrival	Departure	

Secunderabad	--	10:00	
Kazipet	11:30	11:35	
Warangal	12:10	12:12	
Vijayawada	14:00	14:05	
Nellore	16:00	16:02	
Chennai	19:00	--	

Real-Time Train Run	ETA Engine
Status could be: NOT_STARTED RUNNING AT_STATION DELAYED CANCELLED COMPLETED	<pre>interface EtaProvider { EstimatedArrival estimate(VehicleState vehicle, TransportStop stop); }</pre> Implementations - ScheduledEtaProvider - GPSBasedEtaProvider - HistoricalEtaProvider - TrafficAwareEtaProvider - RailwayOperationalEtaProvider

For trains, you could eventually have:

RailwayOperationalEtaProvider

Delay Calculation Compare: Scheduled Arrival Against: Predicted Arrival Example: Scheduled: 14:20 Predicted: 14:35 Delay: 15 minutes	Operator Integration Different transport operators may expose different data formats. <pre>interface OperatorAdapter { TransportData importData(); }</pre> Implementations - OperatorApiAdapter - GtfsAdapter - CsvOperatorAdapter - RailwayDataAdapter - MetroDataAdapter - RailwayOperationalEtaProvider
---	---

